

Google Docs Architecture

Real-Time Collaborative Editing - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	What is Google Docs? Requirements	What must the system do?
2	Capacity Estimation, Operations and Snapshots	How big is the problem?
3	Conflict Resolution: Operational Transformation	How do simultaneous edits not collide?
4	Operational Transformation: Six Worked Cases	How exactly are positions adjusted?
5	Cursor Position Sync	How do cursors stay correct?
6	End-to-End Architecture	How do all components fit together?
7	Failover, Cheat Sheet and Interview Q&A	What if the leader dies? What to remember?

1. What is Google Docs?

Google Docs is a **real-time collaborative document editing system**. It lets **multiple users create, edit and share documents**; changes appear **instantaneously**, people work **simultaneously without conflict**, and the document can be **accessed from anywhere**. The hard part is the word 'without conflict': in a multi-user environment two people can change the same text at the same moment.

EXAMPLE

You and a friend open the same project report. You fix a typo in the title while she adds a paragraph at the bottom. Both edits show up on both screens within a blink, and neither person's work is lost or scrambled.

1.1 Functional requirements

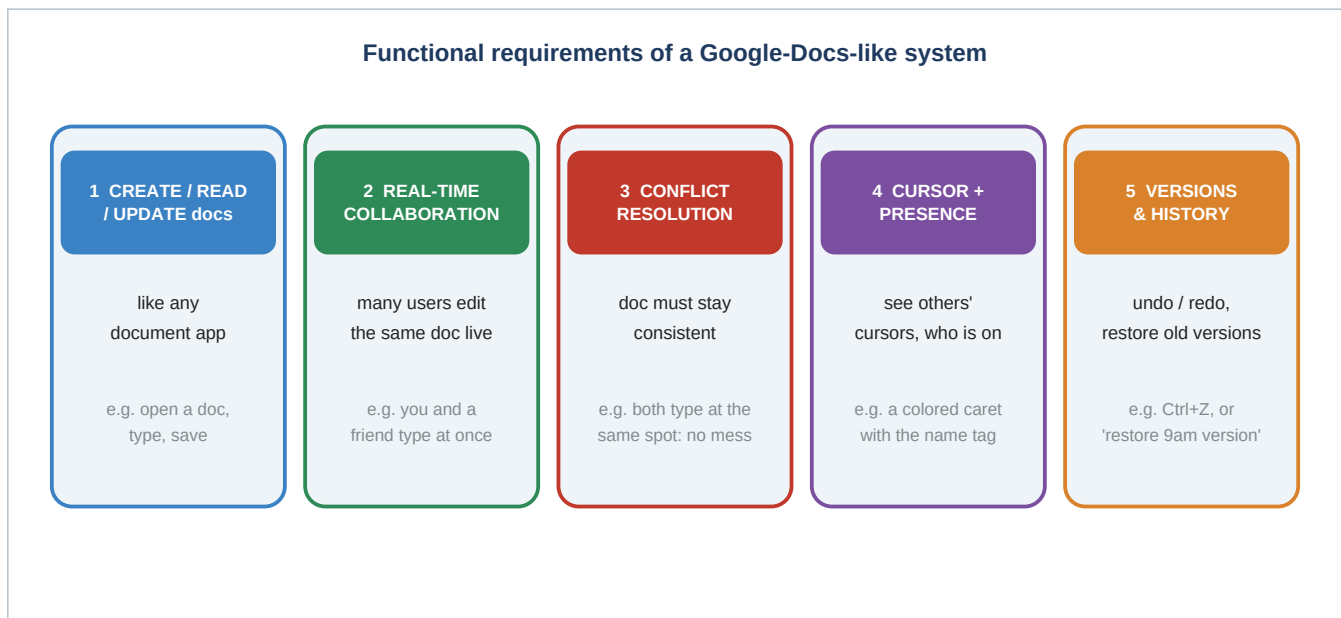


Figure 1: The five functional requirements

- **Create, read and update documents** (basic document management).
- **Real-time collaborative editing:** multi-user support. Example: five teammates type in one meeting-notes doc at once.
- **Conflict resolution:** if multiple users edit simultaneously the doc must not enter a conflicting state; it must **always remain consistent**. Example: two people typing at the same position must not produce garbled text.
- **Cursor presence:** cursors must stay synchronized and the system tracks **who is present**. Example: you see your colleague's colored cursor with her name tag moving in the paragraph she is editing.
- **Version and history:** so we can perform **undo and redo** (and view or restore older versions). Example: press Ctrl+Z, or restore yesterday's version after a bad edit.

KEY TAKEAWAY

The hardest requirement is conflict resolution. The whole architecture is built around an **Operational Transformation (OT)** layer that guarantees consistency.

2. Capacity Estimation, Operations and Snapshots

2.1 Why estimate capacity first?

If you do not know how many users will use the service, you cannot design it. For two or 100 users a plain client and server are enough. For **millions** of users you need optimizations and system design techniques. So capacity estimation comes **before** the design, and you should **confirm assumptions with the interviewer**.

EXAMPLE

Assume 2 documents per user. If the interviewer says 'make it 5', your storage estimate goes up 2.5x and your choice of storage may change. Estimating early avoids picking storage that is not optimal.

2.2 Traffic and write throughput

- **Daily active users (DAU):** 100 million.
- **Peak concurrent users:** 1% of DAU = **1 million** users at the same time.
- **Typing speed:** 5 characters per second per user. Each typed character is one **operation**, so 5 operations/s/user.
- **Total:** 1,000,000 users x 5 ops/s = **5 million operations per second**. The system must handle this.

2.3 Size per operation and write bandwidth

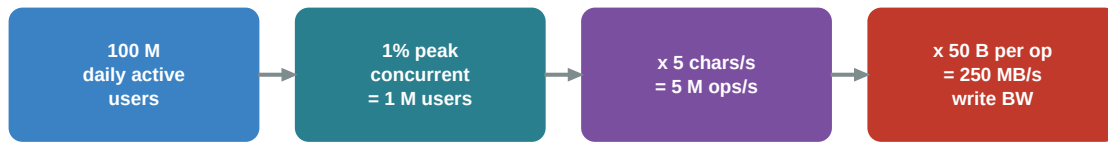
- An operation is compressed JSON-like metadata of about **50 bytes**.
- Write bandwidth = 5,000,000 ops/s x 50 B = 250,000,000 B/s = **250 MB/s** of incoming traffic.
- So this is a **write-intensive** application.

2.4 Storage

- Average document size: **100 KB**. Documents per user: **2** (assumed).
- Documents: 100 M users x 2 = **200 million**.
- Raw storage: 200 M x 100 KB = **20 TB**.
- Version history assumed to be **5x** growth, so about **100 TB** in total.

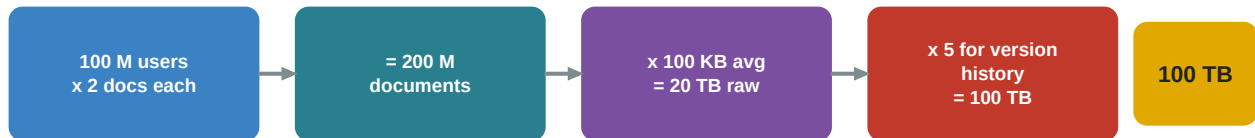
Capacity estimation, step by step

A) Write throughput and bandwidth



=> a WRITE-INTENSIVE system: every keystroke is an operation

B) Storage



Always confirm assumptions with the interviewer (e.g. 5 docs per user instead of 2)
and re-compute: the design must follow the numbers

Figure 2: Throughput, bandwidth and storage estimation

EXAMPLE

Quick sanity check of the math: $200,000,000 \times 100,000 \text{ B} = 20,000,000,000,000 \text{ B} = 20 \text{ TB}$. Multiplying by 5 for history gives 100 TB. Each of these numbers drives a design choice: 5M ops/s means we need a distributed, write-friendly pipeline (WebSocket servers, Kafka); 100 TB means a scalable persistent store.

2.5 What an operation looks like

Every key press (insert or delete) is an **operation**: a small JSON-like record containing the type of edit, the position, the value, and the **version** of the document the client had seen.

Every key press becomes a small JSON operation (~50 bytes)

You do	Operation (JSON-like)	Document after
type a	<code>{"type": "insert", "pos": 0, "val": "a", "v": 0}</code>	a
type b	<code>{"type": "insert", "pos": 1, "val": "b", "v": 1}</code>	ab
type c	<code>{"type": "insert", "pos": 2, "val": "c", "v": 2}</code>	abc
delete c	<code>{"type": "delete", "pos": 2, "len": 1, "v": 3}</code>	ab

pos = index in the text | val = the character | len = how many chars (used by delete)

v = version the client had seen when it made the edit (the 'base version')

Operations are stored, so undo / redo and history come for free

Figure 3: Typing a, b, c and deleting c produces four operations

EXAMPLE

Type a, b, c: insert 'a' at index 0 (v0), insert 'b' at index 1 (v1), insert 'c' at index 2 (v2). Then delete 'c': delete at index 2, length 1 (v3). The document is now 'ab'. The server stores all four operations, so history and undo are straightforward.

2.6 Snapshots: avoiding long replays

A **snapshot** is a **full copy of the document at a specific state**. Suppose a document has received **10,000 operations**. If we stored only operations and the server crashed, rebuilding the document would mean **replaying all 10,000**: not optimal. Instead we store a **full snapshot every Nth operation** (assume every 100).

Snapshots: store a full copy every N operations to avoid long replays

WITHOUT snapshots: server crashes, replay ALL 10,000 operations



op 1 ... op 10,000 (slow, not optimal)

WITH snapshots (e.g. one every 100 ops): load the latest one, replay only the tail



latest snapshot (v1000)

replay only these (≤ 100)

Recovery recipe: 1) load the latest snapshot from the database (e.g. state 'hello' at v1000)
2) fetch the remaining operations after v1000 from Kafka 3) replay them 4) current state restored quickly

Figure 4: Snapshots turn a 10,000-operation replay into a replay of at most ~100

EXAMPLE

A snapshot exists at version 1000 and the document says 'hello' at that point. The server crashes after version 1070. Recovery: load the v1000 snapshot from the database, fetch operations 1001-1070 from Kafka, replay those 70 operations, and the document is back instantly.

KEY TAKEAWAY

Snapshots + an operation log = fast recovery. The DB stores snapshots and logs; Kafka supplies the recent operations.

3. Conflict Resolution: Operational Transformation

This is the **core and heart** of the Google Docs architecture. When many users edit simultaneously the document must not enter an inconsistent state. The technique is **Operational Transformation (OT)**.

3.1 The problem

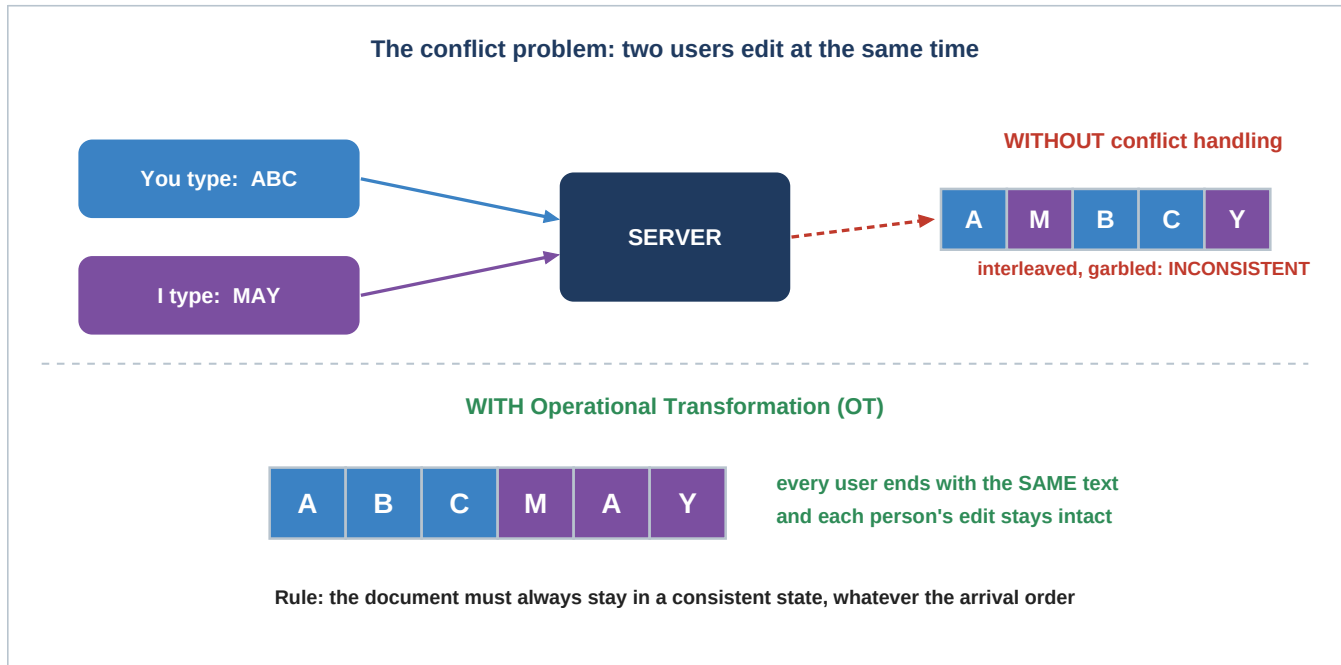


Figure 5: Without conflict handling edits can interleave into garbage; OT keeps them intact

EXAMPLE

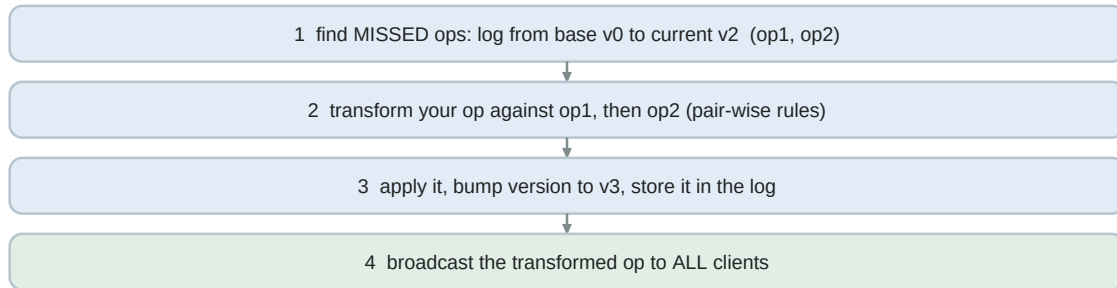
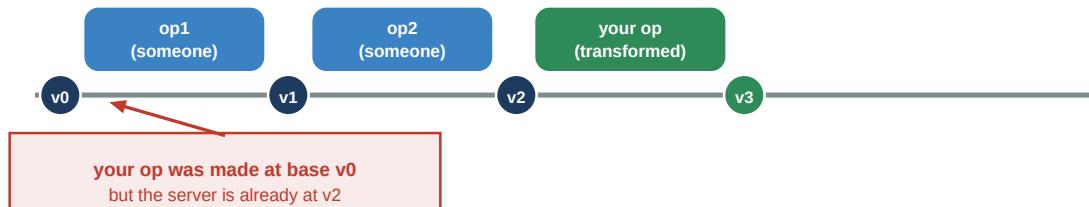
You type ABC while I type MAY at the same time. The correct result for everyone is **ABCMAY**. It must never end up as 'AMBCY' (letters interleaved), and different users must never see different texts.

3.2 Core idea

- The **server keeps a version log** of operations: every operation gets a version number.
- Each incoming operation carries a **base version**: the version of the document the client had seen when it made the edit.
- If the server is already ahead, the operation is **transformed against all operations applied after its base version**, using **pair-wise transformation rules**: insert-insert, insert-delete, delete-insert, delete-delete.
- After transformation the operation is **applied, the version is incremented and it is broadcast to all clients**.

Operational Transformation on the server: transform against what you missed

Server version log



Pair-wise rules cover: insert-insert, insert-delete, delete-insert, delete-delete

Figure 6: A late operation is transformed against the operations it missed

EXAMPLE

You send 'A' with base v0. By the time it reaches the server, my 'MAY' operations have already been applied (server at v2... then v3 after MAY). The server sees your base version is older, takes the operations you missed, transforms your insert so it lands after them, applies it, bumps the version and broadcasts the final document to all.

3.3 What the operation and the server store

What an operation carries and what the server remembers

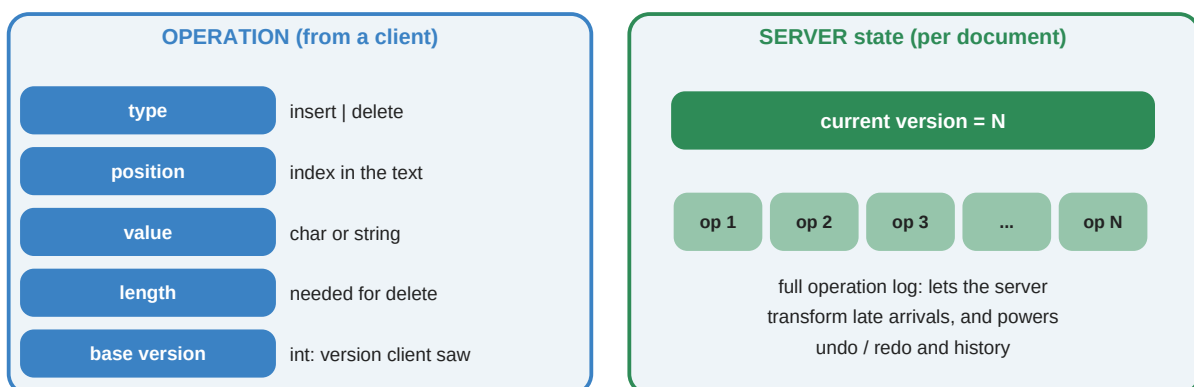


Figure 7: Operation fields and the per-document server state

- **type:** insert or delete.
- **position:** where in the text.
- **value:** a char or string (for insert).
- **length:** how many characters (needed for delete).
- **base version:** an integer, the version the client was at.
- **Server side:** the **current version N** and the **log of operations 1..N**. A late operation is transformed against these logged operations.

3.4 High-level algorithm for an incoming operation

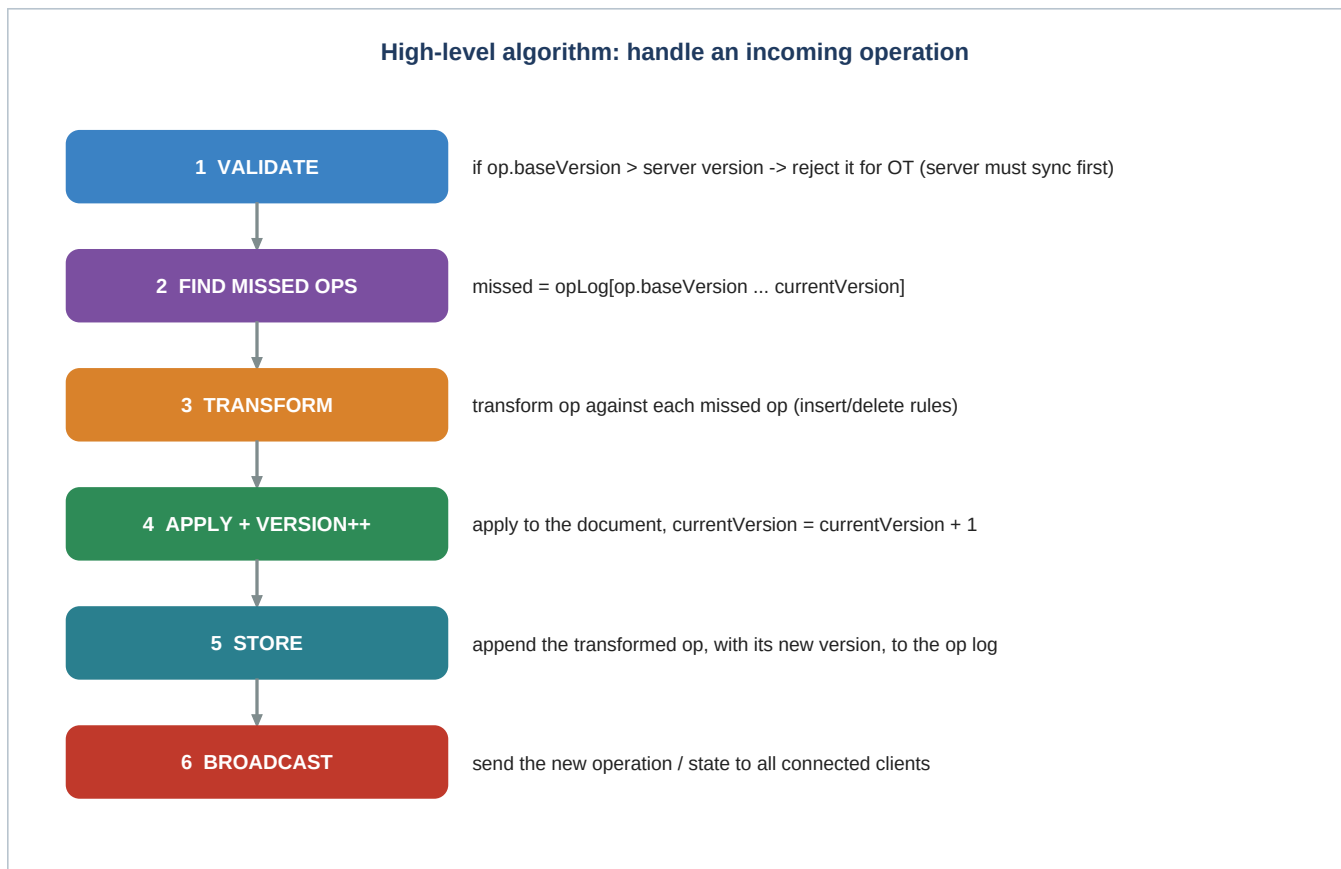


Figure 8: Handling an incoming operation, step by step

- **Validate:** if the operation's base version is **greater than** the server's version, reject it for OT (not rejecting the request entirely; the server first needs to sync up to the client's state).
- **Find missed operations:** the operation log from the base version up to the current version.
- **Transform** the operation against each missed operation.
- **Apply and update version:** increase the current version by one.
- **Store** the new operation with its new version.
- **Broadcast** to all clients.

4. Operational Transformation: Six Worked Cases

In every example the **initial document** is 'abcde' (or 'abcdefg') at **version v0**. Two users, A and B, act on v0 at the same time. The server receives **A first, then B**, so A is applied directly and B must be transformed. The number of 'missed operations' for B is 1 (A's operation).

4.1 Case 1: Insert vs Insert

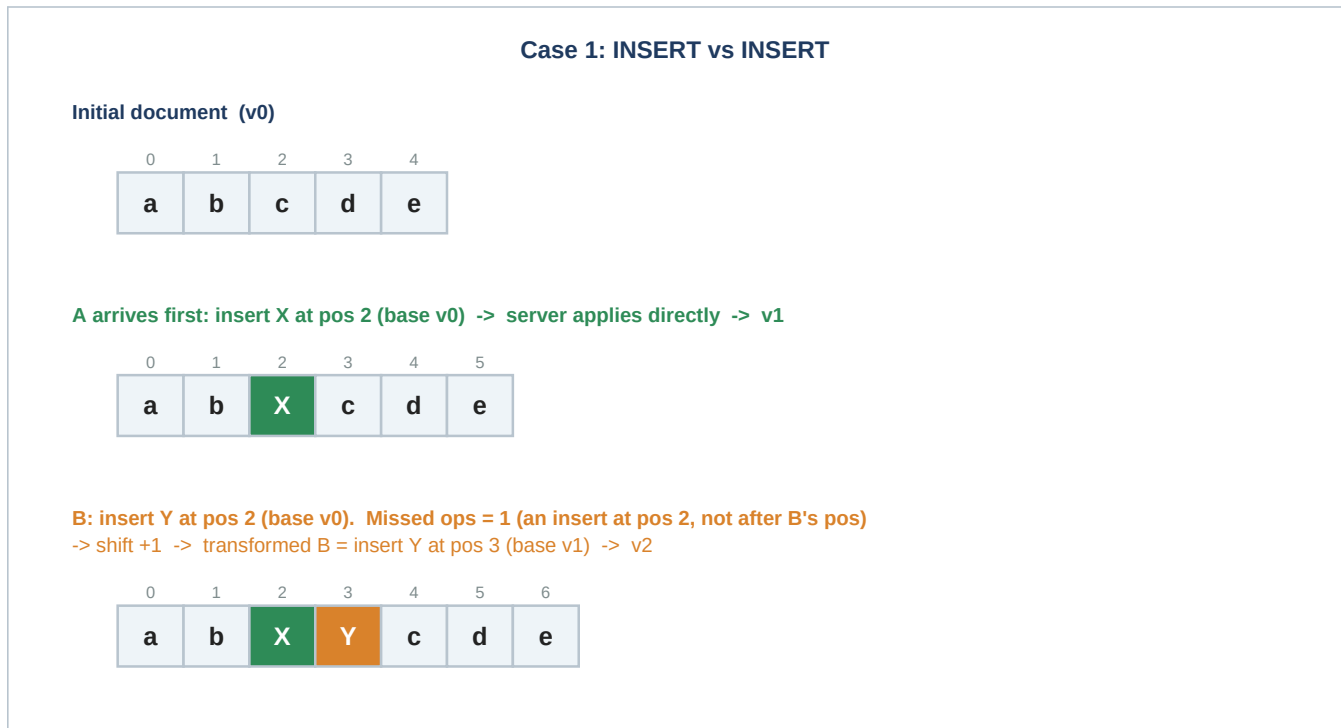


Figure 9: Both users insert at position 2

EXAMPLE

A: insert X at pos 2 (v0). B: insert Y at pos 2 (v0). After A the doc is abXcde (v1). B's position 2 is where X now is, so B shifts right by the number of missed inserts: $\text{pos } 2 + 1 = 3$. Final: **abXYcde**, v2. Both letters survive and the order is decided by arrival order.

4.2 Case 2: Delete then Insert

Case 2: DELETE then INSERT

Initial document (v0). A will delete 'b' (pos 1); B will insert X at pos 3

0	1	2	3	4
a	b	c	d	e

A arrives first: delete pos 1 ('b') -> v1

0	1	2	3
a	c	d	e

B: insert X at pos 3 (base v0). Missed ops = 1 (a DELETE before pos 3)

-> shift -1 -> transformed B = insert X at pos 2 -> v2

0	1	2	3	4
a	c	X	d	e

Figure 10: A deletes before B's insert position

EXAMPLE

A: delete 'b' at pos 1. B: insert X at pos 3 (v0). After A the doc is acde (v1): everything after 'b' moved one place left. So B's position is **decreased by 1**: pos 3 - 1 = 2. Final: **acXde**, v2. B wanted X between 'c' and 'd', and it still is.

4.3 Case 3: Insert then Delete

Case 3: INSERT then DELETE

Initial document (v0). A will insert X at pos 2; B will delete 'd' (pos 3)

0	1	2	3	4
a	b	c	d	e

A arrives first: insert X at pos 2 -> v1

0	1	2	3	4	5
a	b	X	c	d	e

B: delete pos 3 (base v0). Missed ops = 1 (an INSERT before pos 3)

-> shift +1 (right shift) -> transformed B = delete pos 4 (still 'd') -> v2

0	1	2	3	4
a	b	X	c	e

Figure 11: A inserts before B's delete position

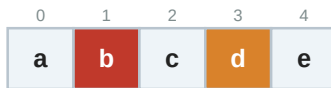
EXAMPLE

A: insert X at pos 2. B: delete the char at pos 3 ('d') (v0). After A the doc is abXcde (v1): 'd' is now at index 4. An insert before the delete means a **right shift**: delete pos $3 + 1 = 4$. Final: **abXce**, v2. Without the shift B would have deleted 'c', the wrong character.

4.4 Case 4: Delete vs Delete

Case 4: DELETE vs DELETE

Initial document (v0). A deletes 'b' (pos 1); B deletes 'd' (pos 3)



A arrives first: delete pos 1 -> v1



B: delete pos 3 (base v0). Missed ops = 1 (a DELETE before pos 3)

-> shift -1 (left shift) -> transformed B = delete pos 2 ('d') -> v2



Figure 12: Two deletes: the earlier one shifts the later one left

EXAMPLE

A: delete 'b' at pos 1. B: delete 'd' at pos 3 (v0). After A the doc is acde (v1), 'd' is at index 2. A delete before a delete means a **left shift**: $\text{pos } 3 - 1 = 2$. Final: **ace**, v2. Both intended characters are gone, no extra character is removed.

4.5 Case 5: Delete range, then Insert outside the range

Case 5: DELETE RANGE then INSERT outside the range

Initial document (v0). A: delete 3 chars from pos 2 (c,d,e). B: insert X at pos 6

0	1	2	3	4	5	6
a	b	c	d	e	f	g

A arrives first: delete range (pos 2, len 3) -> v1

0	1	2	3
a	b	f	g

B: insert X at pos 6 (base v0). Pos 6 is AFTER the deleted range [2..4]

-> subtract the deleted length (6 - 3) -> transformed B = insert X at pos 3 -> v2

0	1	2	3	4
a	b	f	X	g

Figure 13: Insert point lies after the deleted range

EXAMPLE

Doc abcdefg (v0). A: delete 3 characters starting at pos 2 (c, d, e). B: insert X at pos 6 (between f and g). After A: abfg (v1). Pos 6 is after the deleted range, so subtract the deleted length: $6 - 3 = 3$. Final: **abfXg**, v2. X still sits between f and g.

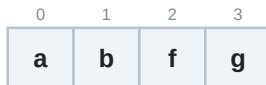
4.6 Case 6: Insert inside a deleted range

Case 6: INSERT inside a deleted range

Initial document (v0). A: delete 3 chars from pos 2. C: insert X at pos 3 (INSIDE that range)



A arrives first: delete range (pos 2, len 3) -> v1



C: insert X at pos 3 (base v0). Pos 3 falls INSIDE the deleted region

-> move to the START of the deleted region -> transformed C = insert X at pos 2 -> v2

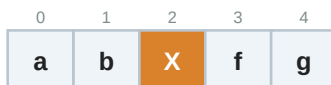


Figure 14: Insert point falls inside the deleted range

EXAMPLE

Same A as before (delete c, d, e). Now C: insert X at pos 3, which was inside the region A deleted. That spot no longer exists. Rule: **move the insert to the start of the deleted region** (pos 2). Final: **abXfg**, v2.

4.7 The rules in one table

Operation applied first (missed)	Incoming operation	Adjustment to the incoming position
Insert before (or at) the position	Insert or Delete	Shift right: + length of the insert
Delete before the position	Insert or Delete	Shift left: - length of the deleted range
Delete range, insert point AFTER the range	Insert	Subtract the deleted length
Delete range, insert point INSIDE the range	Insert	Move to the start of the deleted range

GOOD TO KNOW

The video demonstrates the common, simplified cases. Real OT implementations also handle extra edge cases, for example two users deleting the *same* character (the second delete becomes a no-op) or partially overlapping delete ranges, and they use a deterministic tie-break when positions are equal (here: arrival order at the server).

4.8 Why the server stores the operations

Because the server keeps all operations (op 1, op 2, ...), you can easily implement **undo**, build **version history** and rebuild a document after a failure (together with snapshots).

5. Cursor Position Sync

Positions matter not only for text but also for **cursor**s. If someone inserts text before your cursor and your cursor index stays the same, it ends up in the wrong place. The solution: **apply the same operational transformation to cursor positions as to operations**. When a remote edit occurs before the cursor, the cursor is shifted so it keeps the user's intended position.

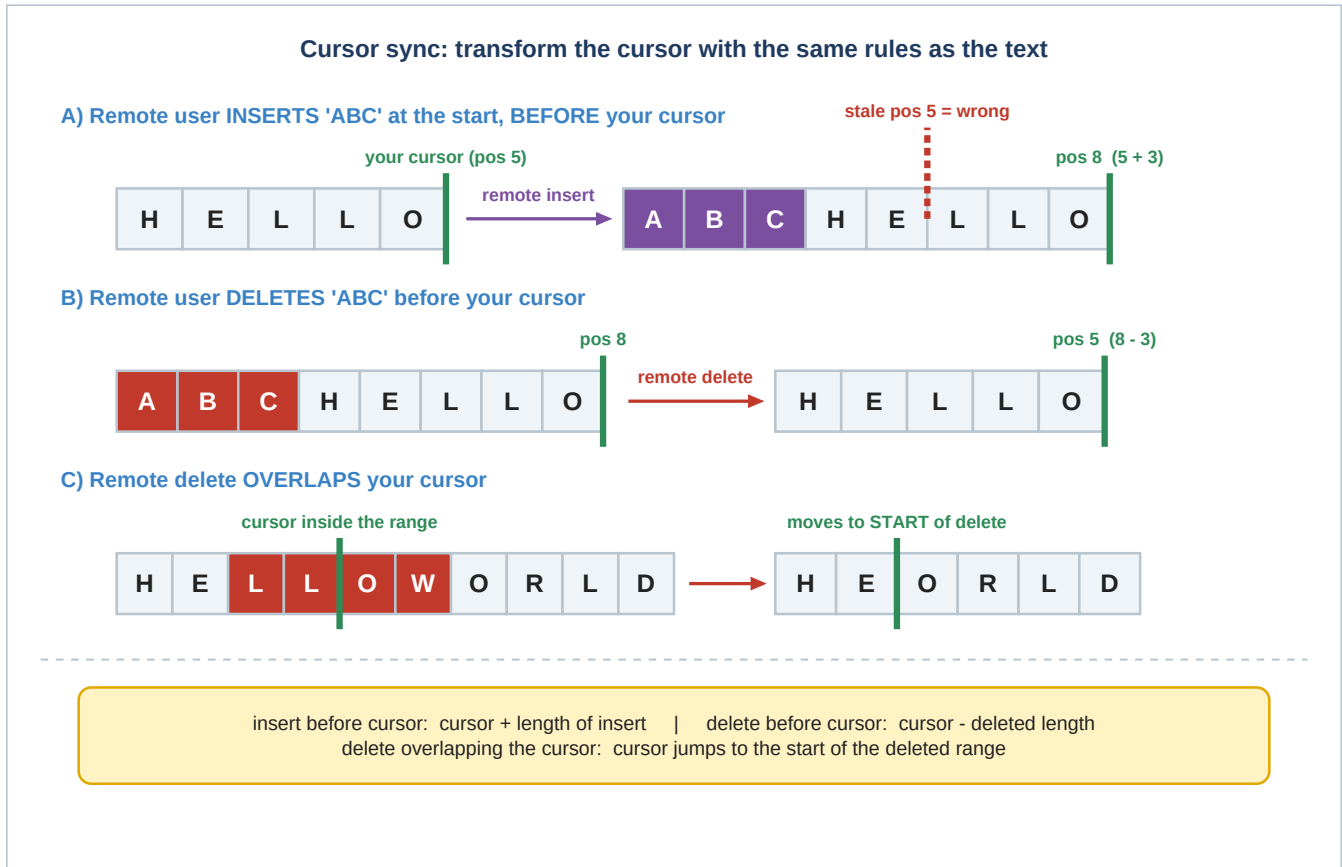


Figure 15: The cursor is transformed with the same rules as the text

Remote edit	Cursor update	Example
Insert BEFORE the cursor	$\text{cursor} = \text{cursor} + \text{length of insert}$	'HELLO' with cursor at 5; remote inserts 'ABC' at start -> cursor 8
Delete BEFORE the cursor	$\text{cursor} = \text{cursor} - \text{deleted length}$	'ABCHELLO' cursor 8; remote deletes 'ABC' -> cursor 5
Delete OVERLAPS the cursor	cursor moves to the START of the deleted range	cursor inside the deleted 'LLOW' -> jumps to where the deletion began
Edit AFTER the cursor	no change	remote types at the end of the doc; your caret stays

EXAMPLE

Without this, you type 'HELLO', a colleague adds 'ABC' at the beginning, and your caret would jump into the middle of 'ABCHELLO' while you were about to type the next letter. With cursor OT your caret stays right after the 'O', exactly where you meant it, and her caret stays correct on your screen too.

KEY TAKEAWAY

Same OT logic for text and cursors: positions are always correct and always synchronized across users.

6. End-to-End Architecture

In an interview you start from this architecture and, when asked about the collaboration layer, dive into the OT logic above. Here is the full picture and then every component in detail.

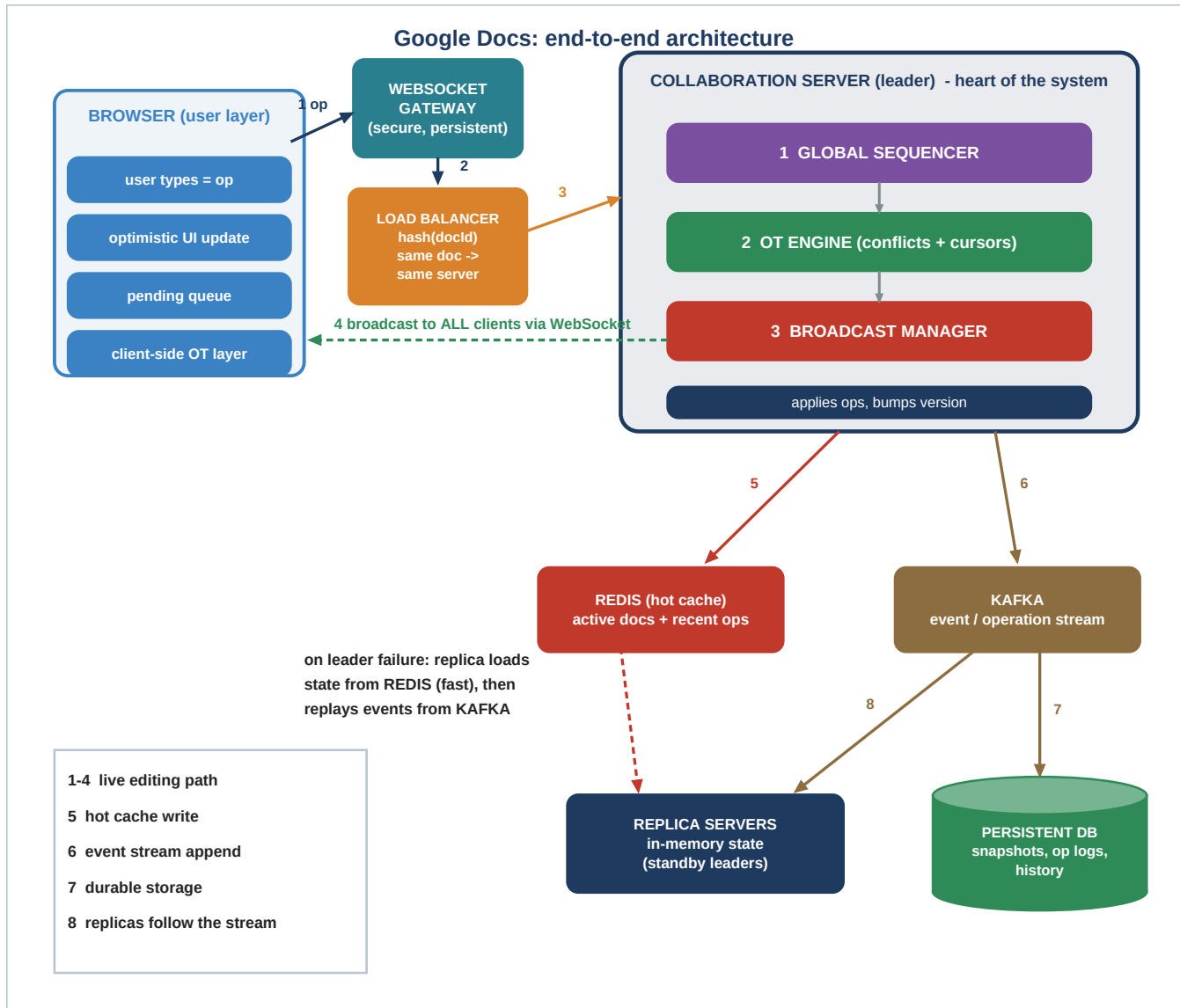


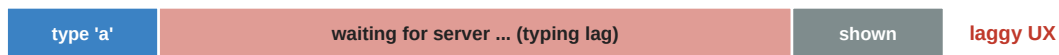
Figure 16: Google Docs end-to-end architecture

6.1 User layer (browser): operations and optimistic UI

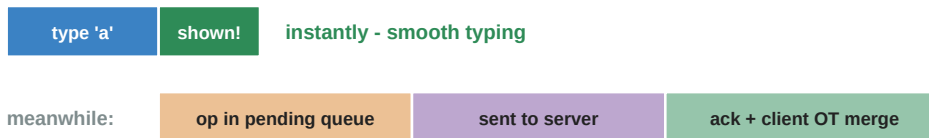
When the user types, an **operation is created on the client**. Example: typing 'a' at position 10 creates insert 'a' @10, base version 10 as a JSON operation.

Optimistic UI update: show the keystroke first, confirm with the server later

WITHOUT optimistic update (wait for the network round trip)



WITH optimistic update (what Google Docs does)



The client keeps a pending queue and its own small OT layer that reconciles local edits with the operations the server broadcasts back

Figure 17: Optimistic update: show the keystroke immediately, reconcile with the server later

- **Optimistic UI update:** the UI applies the change **immediately**, then sends it to the server. The reason is **no typing lag**: the user experience stays smooth.
- The operation is added to a **pending queue**.
- A **client-side OT layer** tracks changes coming from the server and synchronizes them with the local pending operations. It is specific to the client; the server's OT layer handles global ordering.

EXAMPLE

On a slow hotel Wi-Fi with 300 ms latency, waiting for the server before showing each letter would make typing feel broken. With optimistic updates the letter appears at once; confirmation arrives later and, if others edited meanwhile, the client quietly adjusts.

6.2 WebSocket gateway: the real-time operation stream

Operations travel through a **secure WebSocket gateway**. A WebSocket is a **persistent, bidirectional** connection, so the client can send operations and the server can push other users' operations at any moment, with no repeated polling. The gateway maintains the persistent connections and forwards messages onward to the load balancer.

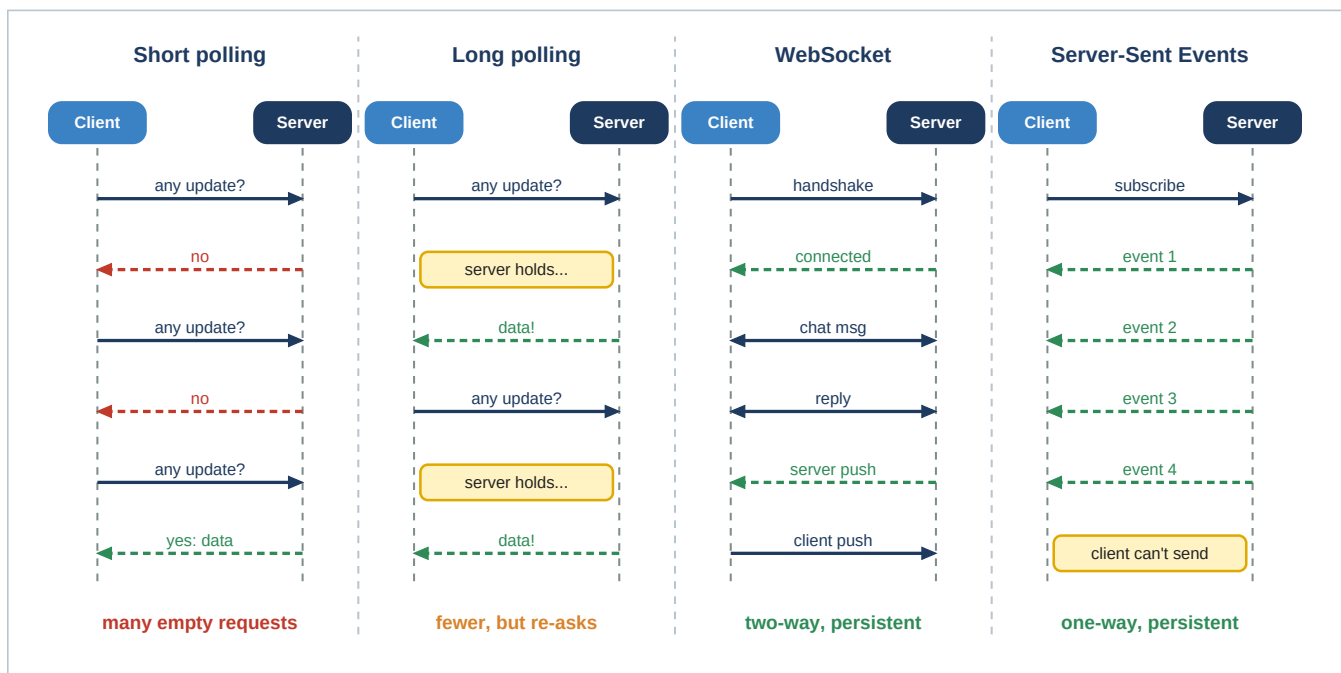


Figure 18: Why WebSocket: persistent two-way connection vs polling

EXAMPLE

Your colleague types a word. The server pushes it to your open socket in milliseconds. With polling, your browser would have to ask 'anything new?' repeatedly, wasting requests and adding delay.

6.3 Load balancer: same document, same collaboration server

The load balancer ensures that **all traffic for one document always goes to the same collaboration server**. It **hashes the document ID** to find the shard.

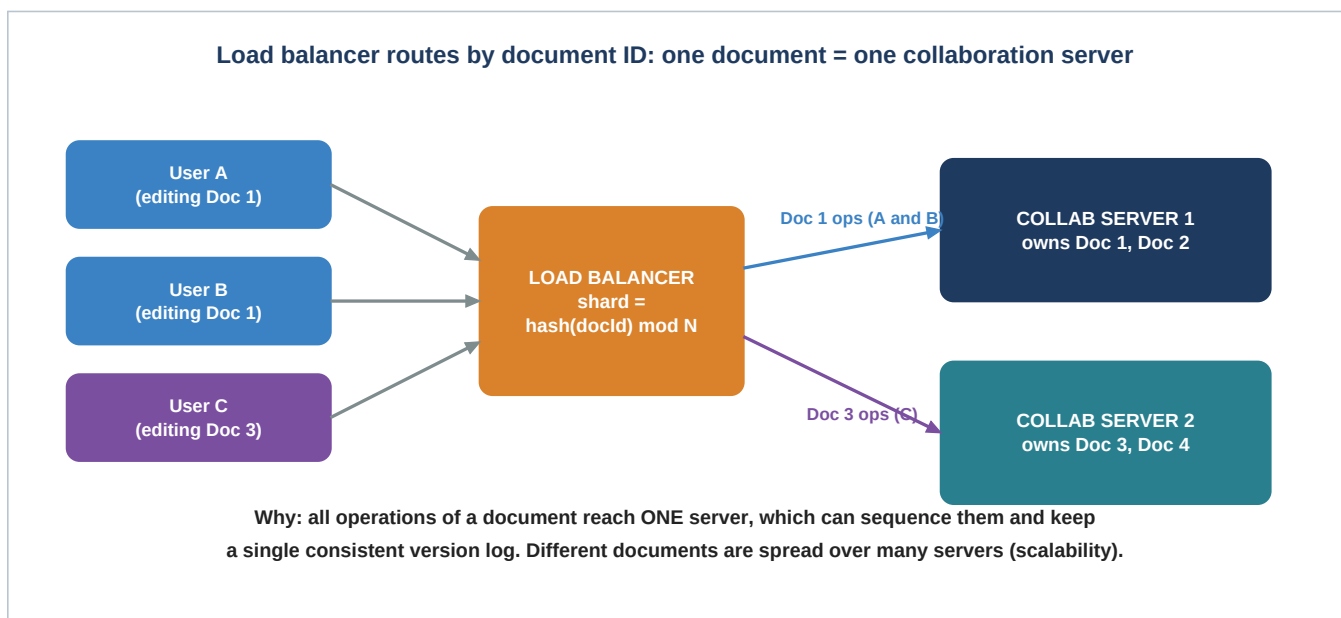


Figure 19: Hashing the document ID keeps every document on a single collaboration server

EXAMPLE

Documents 1 and 2 are handled by shard 1. When User A and User B both edit Doc 1, both connections are routed to collaboration server 1, which therefore sees every operation for that document and can order them. User C editing Doc 3 is routed to server 2.

6.4 Collaboration server: the heart of the architecture

The collaboration server is the **heart** of the system. It has three jobs, in order:

(1) Global sequencer

Multiple operations (A, B, C) arrive at roughly the same time. The **global sequencer** puts them in **one agreed order** using timestamps and other tie-break parameters, so even operations with the same timestamp get a deterministic order.

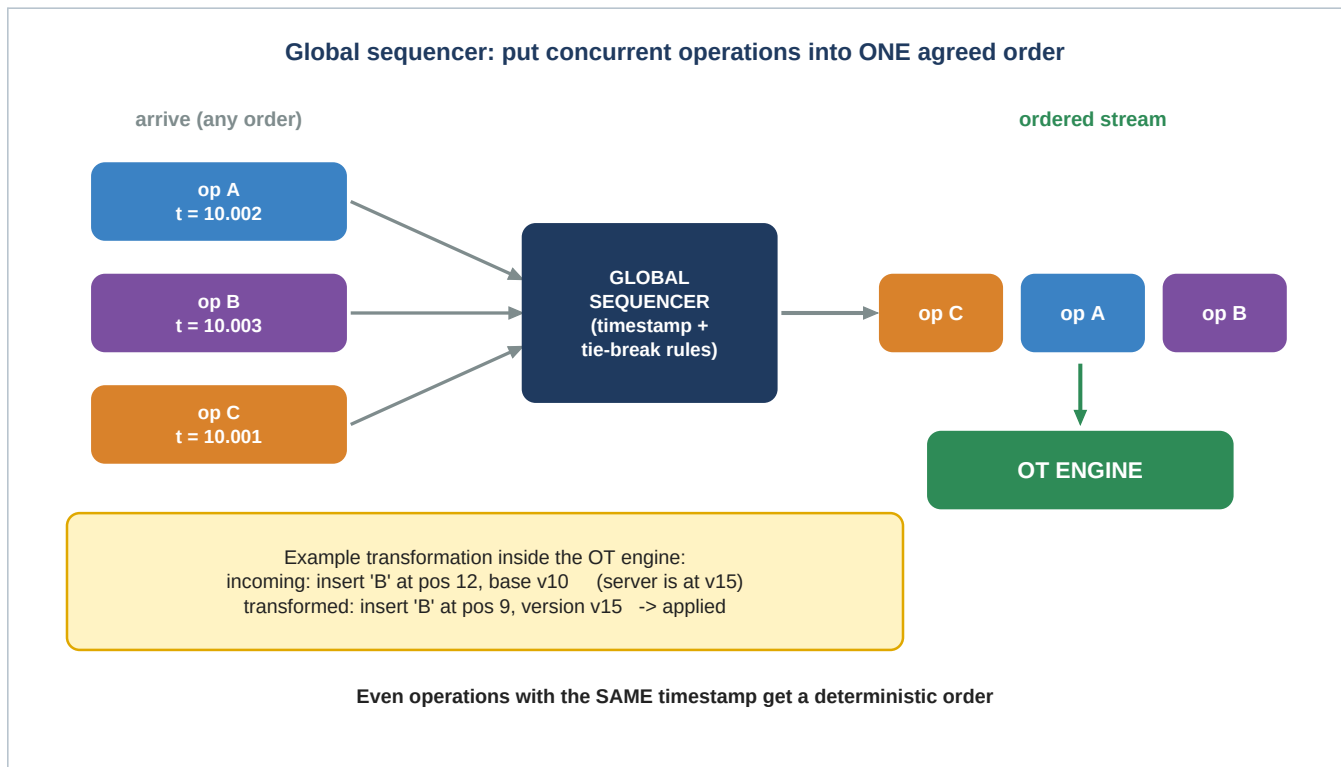


Figure 20: The sequencer orders concurrent operations before the OT engine

EXAMPLE

Operations arrive as A, B, C but the sequencer, using timestamps and tie-breaks, decides the order is C, A, B. The OT engine processes them in exactly that order.

(2) OT engine

The ordered operations go to the **OT engine**, which transforms each one against the operations it missed (insert, delete, delete-range cases) as described in chapters 3 to 5, and also transforms cursors.

EXAMPLE

Incoming: insert 'B' at position 12, base version V10, but the server is at V15. After transformation against the 5 missed operations it becomes: insert 'B' at position 9, version V15 (earlier deletes shifted the position left). It is then applied.

(3) Broadcast manager

Once the operation is applied, the **broadcast manager fans out the update to all clients** through the WebSocket gateway, so every collaborator sees the same result.

6.5 Redis: hot cache

The collaboration server passes operations to **Redis**, a **hot cache**. It stores the **active documents** and the **recent operations after the last snapshot**, for very fast access (for example to recover state quickly).

EXAMPLE

A document that 20 people are editing right now lives in Redis along with the operations since its last snapshot; a document nobody has opened for a month is not in Redis, only in the persistent store.

6.6 Kafka: the operation event stream

The operation events are also published to **Kafka**, a durable event stream. Kafka feeds two consumers: the **persistent storage** and the **replica collaboration servers**.

6.7 Persistent storage

Persistent storage stores the **snapshots, operation logs and history**. It is what you recover from after a failure and what powers version history.

6.8 Replica collaboration servers

Replicas also consume the events from Kafka and keep an **in-memory state** of documents. They stay consistent with the operations so that they can **act as leader in case of failover**.

6.9 One keystroke, end to end

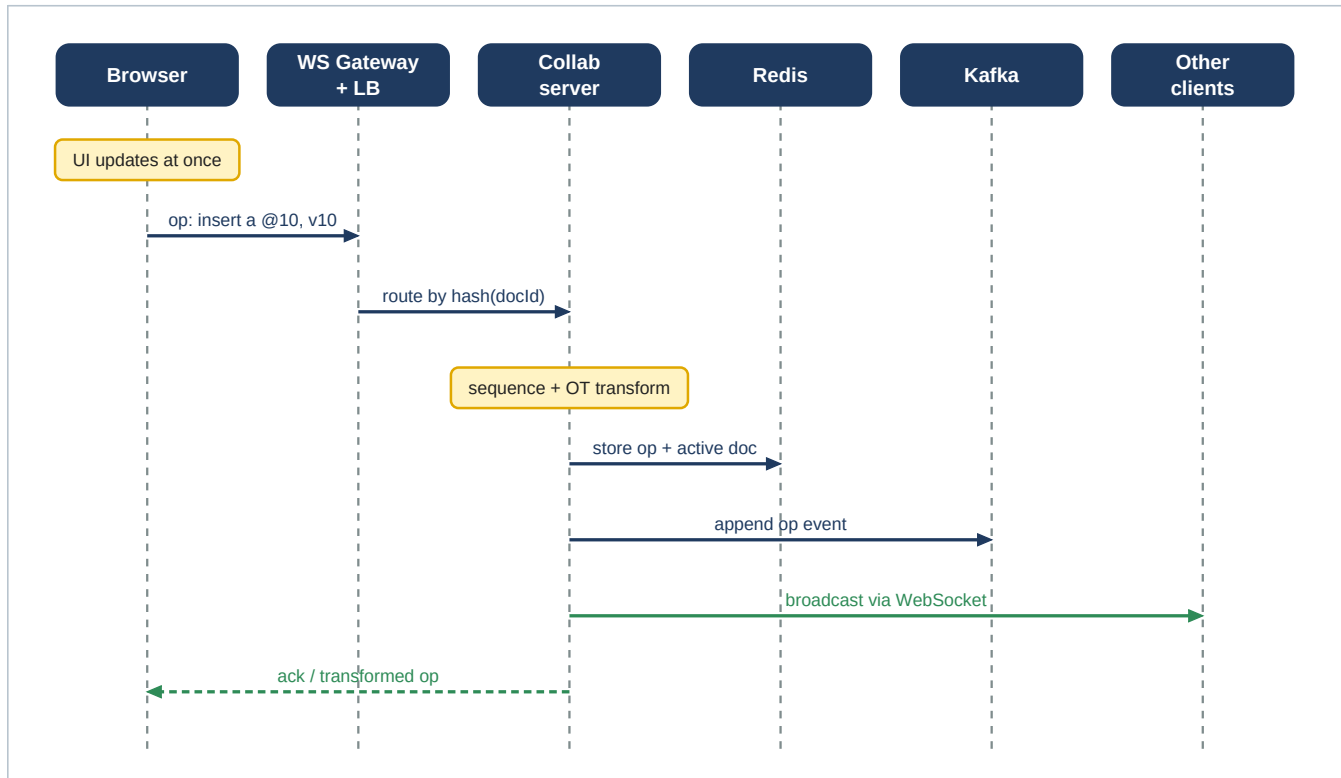


Figure 21: Sequence of a single operation through the system

- 1) Browser applies the edit to the local UI immediately (optimistic) and queues it.
- 2) The operation travels over the secure WebSocket to the gateway / load balancer.
- 3) The load balancer routes it by hash(documentId) to the owning collaboration server.
- 4) The server sequences it and the OT engine transforms it against missed operations; the version is incremented.
- 5) The operation and active document state are stored in Redis.
- 6) The operation event is appended to Kafka (which feeds persistent storage and replicas).
- 7) The broadcast manager pushes the result to all clients via the WebSocket gateway; the sender receives an ack.

Component	Role	Why it is there
Browser + client OT	Create ops, optimistic UI, pending queue	No typing lag; reconcile with server
WebSocket gateway	Persistent, bidirectional real-time stream	Instant push both ways
Load balancer	hash(docId) -> collaboration server	One document, one sequencer
Collaboration server	Sequencer + OT engine + broadcast	Consistency and fan-out
Redis	Active docs + recent ops (hot cache)	Fast reads and fast recovery
Kafka	Durable operation event stream	Feeds storage and replicas, replayable
Persistent DB	Snapshots, operation logs, history	Durability, undo, history
Replica servers	In-memory standby state	Failover without losing the document

7. Failover, Cheat Sheet and Interview Q&A

7.1 What if the leader collaboration server dies?

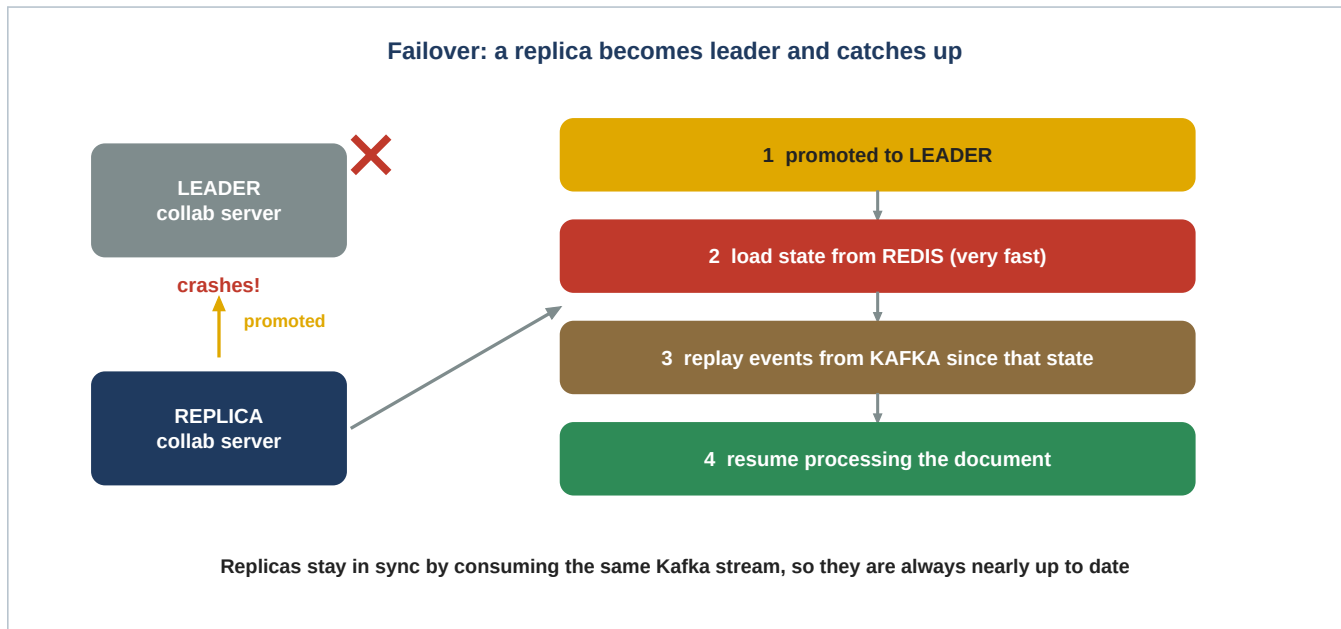


Figure 22: A replica is promoted, loads state from Redis and replays Kafka

- A **replica is promoted to leader**.
- It **loads the state from Redis**, because Redis is very fast.
- It **replays events from Kafka** to reach the latest state of the document.
- It **continues processing** the document.

EXAMPLE

The leader for Doc 1 crashes while 30 people are typing. A replica that has been following Kafka takes over, loads the doc from Redis, replays the last few events and resumes. Users may see a brief pause, but nobody loses their text.

7.2 One-line summary per topic

Topic	Remember this
Goal	Real-time collaborative editing without conflicts, with cursors, undo/redo and history.
Traffic	100 M DAU, 1% = 1 M concurrent, 5 ops/s each = 5 M ops/s (write-intensive).
Bandwidth / storage	50 B per op = 250 MB/s; 200 M docs x 100 KB = 20 TB; x5 history = 100 TB.
Snapshot	Full copy every Nth op; recover = latest snapshot + replay the tail from Kafka.
OT	Server version log; transform late ops against missed ops; apply, bump version, broadcast.
Shift rules	Insert before: right shift. Delete before: left shift. Inside deleted range: move to start.
Cursor	Same transformation applied to cursor positions.
Optimistic UI	Apply locally first, queue, reconcile later: no typing lag.
WebSocket	Persistent bidirectional stream for operations and broadcasts.
Load balancer	Hash document ID so a document always reaches the same collaboration server.
Collab server	Global sequencer -> OT engine -> broadcast manager.
Redis / Kafka / DB	Hot cache / event stream / snapshots + logs + history.
Replicas	In-memory followers; promoted on failure, load from Redis, replay Kafka.

7.3 Interview Q&A

Q: What are the functional requirements?

A: Create/read/update docs, real-time collaboration, conflict resolution, cursor presence, versions and history.

Q: Why do capacity estimation?

A: The design depends on the numbers; clarify assumptions with the interviewer and size storage and throughput accordingly.

Q: How many operations per second?

A: 1 M concurrent users x 5 chars/s = 5 M ops/s; at 50 B each that is 250 MB/s of writes.

Q: How much storage?

A: 200 M docs x 100 KB = 20 TB, times 5 for version history = about 100 TB.

Q: Why snapshots?

A: To avoid replaying thousands of operations: load the latest snapshot and replay only the operations after it.

Q: What is Operational Transformation?

A: A technique that transforms concurrent operations against the ones already applied so all users converge to the same consistent document.

Q: How does the server decide how to transform?

A: It compares the operation's base version with its current version, takes the missed operations from its log and applies pair-wise insert/delete transformation rules.

Q: What happens for insert vs insert at the same position?

A: The one that arrived first stays; the later one is shifted right by one.

Q: What happens for an insert inside a deleted range?

A: It moves to the start of the deleted range.

Q: How is cursor position kept correct?

A: The same OT rules are applied to cursor positions: shift on insert/delete before it, jump to the start if a delete overlaps it.

Q: What is an optimistic UI update?

A: The client applies the edit locally right away, queues it, and reconciles with the server later; it removes typing lag.

Q: Why WebSockets?

A: A persistent bidirectional connection delivers operations in real time without polling.

Q: Why hash the document ID in the load balancer?

A: So every operation of a document hits the same collaboration server, which can sequence them consistently.

Q: What does the collaboration server do?

A: Sequences operations globally, runs the OT engine, applies and versions them, and broadcasts to all clients.

Q: What are Redis and Kafka used for?

A: Redis: hot cache of active docs and recent ops. Kafka: durable operation stream feeding persistent storage and replicas.

Q: What happens if the leader dies?

A: A replica is promoted, loads state from Redis and replays Kafka events, then continues.

7.4 Notes on the source material

- The transcript is auto-generated, so some words are garbled: 'radish / radius' = Redis, 'OT' = Operational Transformation, 'set' in the operation fields = 'insert'.
- In the delete-then-insert example the speaker says the transformed position becomes 'three', but the correct transformed position is **two** (3 minus 1), which is what produces the stated final text 'acXde'. These notes use the correct value.
- The video describes the validation step as rejecting an operation whose base version is greater than the server version 'for operational transformation' while the server syncs; it is not rejecting the edit permanently.
- Numbers (100 M DAU, 5 chars/s, 50 B per op, 100 KB docs, 2 docs per user, 5x history, snapshot every ~100 ops) are the speaker's assumptions for the exercise, not measured figures.
- Real products use more elaborate OT/CRDT machinery; this is the simplified interview-level architecture presented in the video.